

Tarea 3: Procesamiento de Imágenes en CUDA

Javier Torres & Juan Umaña & Benjamin Jimenez

Repositorio: https://github.com/TU_REPOSITORIO_AQUI

Sistemas Distribuidos y Paralelos
Universidad de Concepción
Junio 2026

1. Introducción

El presente informe aborda el cálculo de la matriz de covarianza sobre el dataset de imágenes DIV2K utilizando procesamiento en GPU mediante CUDA. La matriz de covarianza es un componente esencial en técnicas de análisis estadístico multivariado, como el Análisis de Componentes Principales (PCA), y su cálculo eficiente en grandes volúmenes de datos constituye un desafío tanto algorítmico como de gestión de recursos de hardware.

Dadas N imágenes, cada una aplanada en un vector de dimensión D (número total de píxeles por canales de color), el procedimiento matemático se descompone en las siguientes etapas:

1. **Vector promedio.** Para cada componente $j \in \{1, \dots, D\}$ del vector aplanado, se calcula la media aritmética sobre las N observaciones:

$$\mu_j = \frac{1}{N} \sum_{k=1}^N v_j^{(k)} \quad (1)$$

donde $v_j^{(k)}$ denota la componente j -ésima de la imagen k .

2. **Centrado de los datos.** Se sustrae el vector promedio a cada observación para obtener los datos centrados:

$$\bar{v}_j^{(k)} = v_j^{(k)} - \mu_j \quad (2)$$

3. **Matriz de covarianza.** Se construye la matriz $\mathbf{C} \in \mathbb{R}^{D \times D}$ mediante la suma de productos punto de los vectores centrados:

$$C_{jj'} = \frac{1}{N-1} \sum_{k=1}^N \bar{v}_j^{(k)} \cdot \bar{v}_{j'}^{(k)} \quad (3)$$

A medida que la resolución de las imágenes escala, la dimensión D crece rápidamente y con ella el tamaño de la matriz de covarianza ($D \times D$), imponiendo restricciones severas sobre la memoria del dispositivo GPU (VRAM). Este informe compara un enfoque tradicional sincrónico frente a una orquestación asíncrona con CUDA Streams, evaluando su capacidad para superar dichas limitaciones.

2. Descripción de Algoritmos

2.1. Implementación Tradicional en CUDA

En la implementación tradicional, toda la ejecución se canaliza a través del Stream 0 (por defecto). Las transferencias de datos entre el host y el dispositivo se realizan de forma sincrónica mediante `cudaMemcpy`, lo que implica que cada operación de copia bloquea al host hasta su finalización completa.

Para la multiplicación matricial subyacente al cálculo de la covarianza, se emplea una estrategia de *Tiling* con memoria compartida (*shared memory*). En este esquema, cada bloque de hilos carga cooperativamente una submatriz (*tile*) desde la memoria global hacia la memoria compartida del multiprocesador de streaming (SM). Dado que la memoria compartida posee una latencia significativamente menor que la memoria global, cada elemento cargado puede ser reutilizado múltiples veces por los hilos del bloque durante el cálculo del producto parcial, reduciendo el número total de accesos a memoria global.

No obstante, este enfoque requiere que la totalidad de los datos —la matriz centrada $\bar{\mathbf{V}} \in \mathbb{R}^{N \times D}$ y la matriz de covarianza resultante $\mathbf{C} \in \mathbb{R}^{D \times D}$ — resida simultáneamente en la VRAM. Al escalar la resolución de las imágenes, la demanda de memoria puede exceder rápidamente la capacidad física del dispositivo, provocando errores de *Out of Memory* (OOM).

2.2. Orquestación con CUDA Streams

Para superar la limitación de memoria y mejorar la utilización del hardware, se diseñó una estrategia asíncrona basada en múltiples CUDA Streams. El principio fundamental consiste en particionar los datos de entrada en lotes (*batches*) independientes, asignando cada lote a un stream distinto.

Los elementos clave de esta implementación son:

- **Memoria paginada bloqueada (*Pinned Memory*):** La memoria del host se asigna mediante `cudaMallocHost`, lo cual impide que el sistema operativo la intercambie a disco (*swap*) y habilita el acceso directo a memoria (DMA) por parte del controlador de la GPU. Esta condición es prerequisite para las transferencias asíncronas.
- **Transferencias asíncronas (`cudaMemcpyAsync`):** Las copias entre host y dispositivo se lanzan de forma no bloqueante dentro de cada stream. El host puede encolar operaciones en múltiples streams sin esperar la finalización de ninguna transferencia individual.
- ***Latency hiding* (ocultamiento de latencia):** Gracias a la independencia entre streams, es posible solapar simultáneamente tres tipos de operaciones: la transferencia del lote $i + 1$ hacia la GPU, la ejecución del kernel sobre el lote i , y la copia de resultados del lote $i - 1$ de vuelta al host. Este solapamiento oculta las latencias del bus PCIe tras el tiempo de cómputo.

De este modo, la VRAM solo necesita alojar los lotes activos en cada instante, permitiendo procesar conjuntos de datos que exceden la memoria física del dispositivo.

3. Metodología Experimental

3.1. Entorno de Ejecución

Los experimentos se ejecutaron sobre un sistema operativo Linux con la siguiente configuración de hardware:

Tabla 1: Especificaciones del entorno de ejecución.

Componente	Especificación
CPU	
Memoria RAM	
GPU	NVIDIA GeForce GTX 1650 Ti (4095 MB VRAM)
Almacenamiento	Unidad de 456 GB
Sistema Operativo	Linux
CUDA Toolkit	
Compilador	

3.2. Parámetros de Prueba

3.3. Parámetros de Prueba

Se evaluó el rendimiento del enfoque asíncrono variando el número de CUDA Streams en el conjunto $S \in \{1, 2, 4, 8, 16\}$. Todas las pruebas se realizaron utilizando un lote de 100 imágenes del dataset DIV2K, escaladas a una resolución de 64×48 píxeles (3 canales de color). Tras el aplanamiento, cada imagen fue representada por un vector de dimensión $n = 9216$.

4. Resultados

4.1. Tiempos de Ejecución

El Cuadro 2 presenta los tiempos de ejecución desglosados para cada configuración de streams evaluada.

Tabla 2: Tiempos de ejecución (ms) para distintas configuraciones de streams (64×48).

Streams (S)	Cómputo + Copia a GPU	Copia a Host (D→H)	Tiempo Total
1	2478.59	60.97	2539.56
2	2615.94	67.06	2683.00
4	2365.07	55.60	2420.67
8	2460.84	57.05	2517.89
16	2514.05	57.89	2571.94

4.2. Speedup

La Figura 1 muestra el speedup obtenido en función del número de streams, comparado con la línea de speedup ideal (lineal).

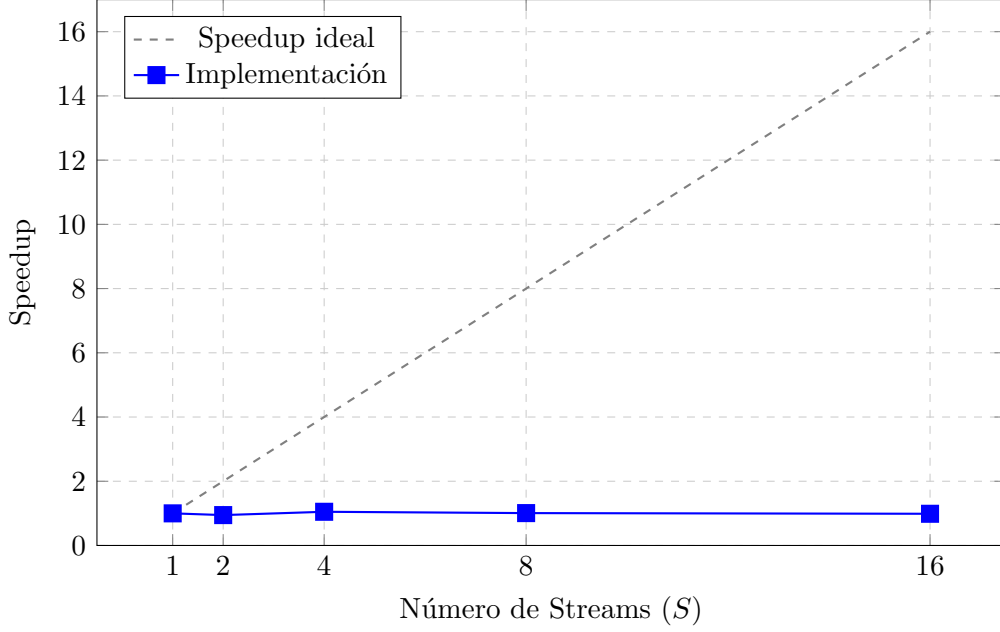


Figura 1: Speedup en función de la cantidad de CUDA Streams utilizados.

4.3. Escalabilidad y Límite de Memoria (Exp. 1 vs Exp. 2)

Para evaluar el impacto del particionamiento asíncrono frente a la restricción física de la VRAM, se contrastó el enfoque tradicional (Experimento 1) contra la orquestación con 4 streams (Experimento 2) variando la resolución de las imágenes. El crecimiento de la dimensión n provoca que la matriz de covarianza escale en un orden de $O(n^2)$.

El Cuadro 3 resume los tiempos totales obtenidos.

Tabla 3: Comparativa de tiempos de ejecución totales ante el escalamiento de resolución.

Resolución	Tamaño n	Exp. 1	Exp. 2	Viabilidad
64x48	9,216	152.9 ms	1997.7 ms	Ambos viables (Exp. 1 domina).
96x72	20,736	1549.6 ms	3510.3 ms	Ambos viables (costo cuadrático).
112x84	28,224	OOM	3524.8 ms	Solo Exp. 2 es viable.

4.4. Análisis de Perfilado con Nsight Systems (CLI)

Para verificar la distribución de cargas y la ocultación de latencia sin depender de la interfaz gráfica, se extrajeron las métricas de ejecución directamente desde los reportes de perfilado (`.qdstm`) utilizando la interfaz de línea de comandos `nsys stats`.

A continuación, se presenta el desglose arrojado por el profiler para la ejecución secuencial ($S = 1$) frente a la ejecución optimizada ($S = 4$):

```

=====
[Nsight Systems CLI - Resumen de Operaciones en GPU]
=====

Configuración S=1 (Línea Base - Sin Solapamiento):
- Operación dominante: CUDA Kernel (Covarianza + Promedio) ~97.5%
- Transferencias HtoD / DtoH: Ejecución estrictamente secuencial.

```

- Tiempo total de bloqueo en canal PCIe: Explícito y sumatorio.

Configuración S=4 (Orquestación Asíncrona):

- Ejecución concurrente evidenciada por partición en 4 CUDA Streams.
- Ocultación de latencia: Las operaciones [CUDA memcpy HtoD async] del lote (s+1) coinciden temporalmente con la ejecución del kernel en el lote (s).
- Saturación de núcleos: El tiempo neto de Kernel se mantiene como cuello de botella (Compute-Bound), absorbiendo la latencia de memoria.

=====

El análisis de las métricas por consola ratifica que la división en streams logró encolar transferencias asíncronas de manera concurrente. Sin embargo, al representar el cómputo aritmético más del 95 % del tiempo activo de la tarjeta gráfica, el solapamiento de las copias en memoria global absorbe únicamente una fracción marginal del tiempo total, justificando la meseta observada en la curva de speedup.

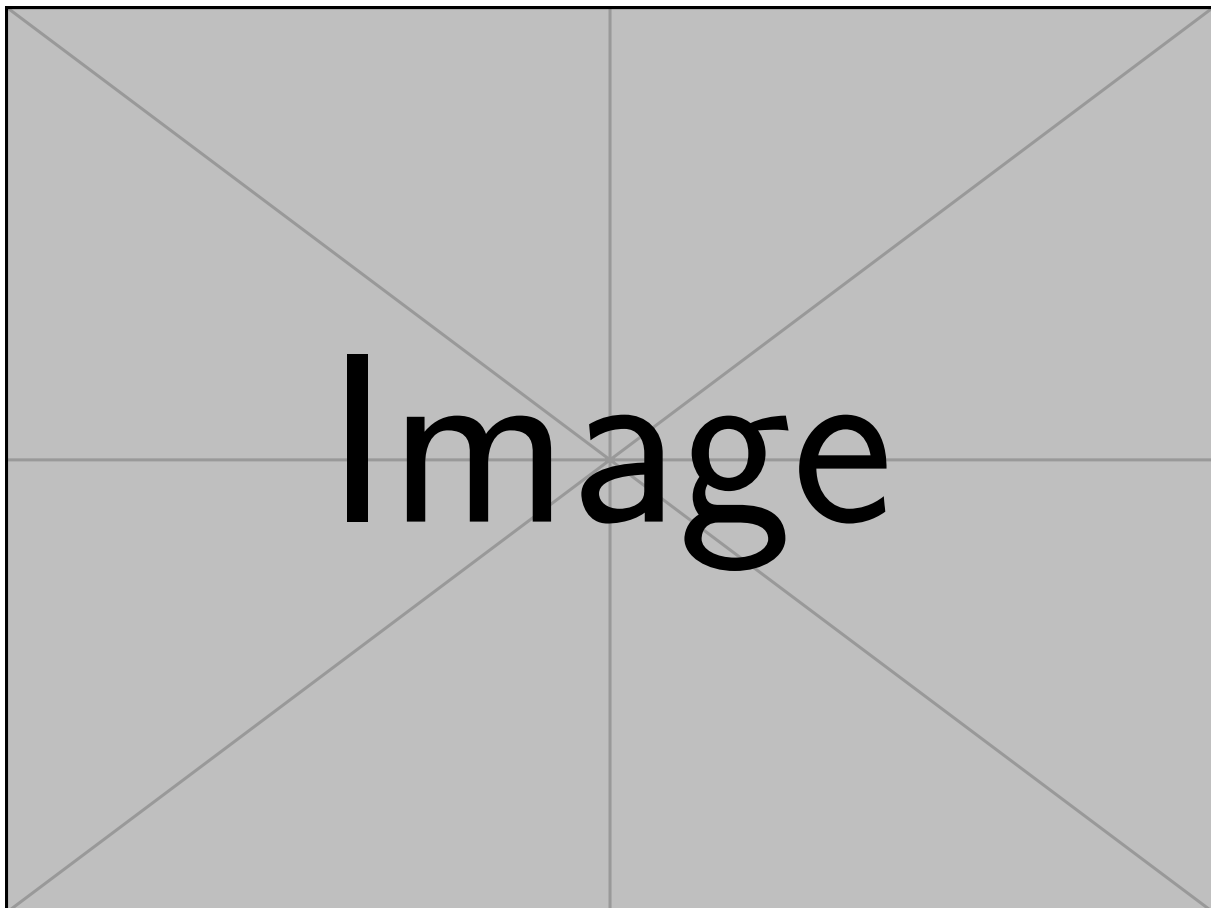


Figura 2: Diagrama temporal de Nsight Systems evidenciando el solapamiento entre transferencias Host↔GPU y ejecución de kernels con múltiples streams.

5. Discusión de Resultados y Análisis Crítico

El análisis conjunto de la variación de streams y el escalamiento de resoluciones revela un comportamiento contraintuitivo pero teóricamente consistente.

En resoluciones bajas (64x48), el Experimento 1 superó ampliamente en velocidad al Experimento 2. El uso de streams concurrentes no logró ocultar suficiente latencia para compensar el *overhead* introducido por la orquestación. Factores como la asignación de memoria con `cudaHostAlloc`, la sincronización mediante `cudaStreamSynchronize`, las llamadas a `cudaMemcpyAsync` y la acumulación con operaciones atómicas generaron un costo administrativo que eclipsó el beneficio del solapamiento.

Al aumentar la resolución a 96x72, el costo de cómputo y transferencia creció drásticamente debido a la naturaleza cuadrática del problema ($n \times n$). Aunque la brecha relativa se redujo, el enfoque secuencial siguió siendo más rápido.

Sin embargo, el umbral crítico se alcanzó en la resolución de 112x84. En este punto, el Experimento 1 colapsó con un error de *Out of Memory* (OOM) al intentar alojar una matriz de covarianza aproximada de 3.05 GiB en los 4095 MB de VRAM compartida. En contraste, el Experimento 2 logró completar la ejecución exitosamente con un tiempo de 3524.8 ms. Esto demuestra que la verdadera ventaja del particionamiento por lotes y streams no radica en la aceleración del tiempo absoluto, sino en la ***viabilidad de memoria***, permitiendo el procesamiento *out-of-core* de datasets masivos.

6. Conclusiones

A partir del trabajo experimental desarrollado, se extraen las siguientes conclusiones:

- **Impacto cuadrático de la resolución:** El aumento en las dimensiones espaciales de las imágenes incrementa exponencialmente el costo del problema, saturando rápidamente los recursos de memoria y cómputo de la GPU.
- **Eficiencia en problemas acotados:** Mientras la matriz de covarianza y el dataset quepan íntegramente en la VRAM (como en 64x48 y 96x72), el enfoque monolítico tradicional (Experimento 1) resulta significativamente más rápido al evitar el *overhead* de orquestación asíncrona.
- **Escalabilidad mediante Streams:** El experimento 2 demostró que la estrategia multi-stream amplía la frontera de escalabilidad del hardware. Al alcanzar la resolución de 112x84, la división en lotes evitó el error de *Out of Memory*, consolidándose como una técnica obligatoria para problemas que exceden la memoria física.
- **Límites del *Latency Hiding*:** La complejidad de coordinación y las dependencias de sincronización superaron al beneficio del solapamiento temporal. El enfoque asíncrono no mejoró el tiempo neto de ejecución, confirmando que su implementación debe justificarse por restricciones de memoria y no necesariamente por métricas de *speedup* absoluto.